Team Mozzarella

MCU: STM32G431CBU6
Flash: 128 KB | RAM: 32KB
Speed: 170 MHz

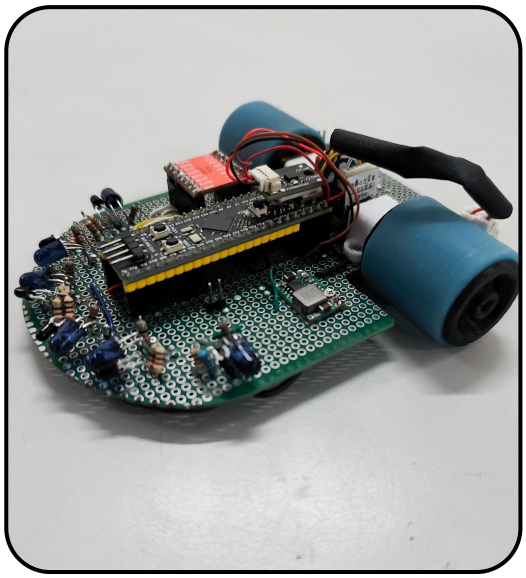
Motors: 12V GA12-N20 Motor (300 rpm motor)
Encoder: Quadrature Hall Sensor pair (3 ppr)

Power: 2S LiPo 350mAh 70C

Sensors: IR sensors (custom pair)
(TSAL6100 IR LED and SFH309 Phototransistor pair)

Motor Driver: Toshiba TB6612FNG
I (continuous): 1.2A per motor

IMU: STMicro. ISM330DHCX
gyro: ±500dps, Communication Interface: I2C



Pascal v0.1

Introduction:

Our micromouse 'Pascal' is a small sized robot designed to navigate and solve mazes efficiently and autonomously.

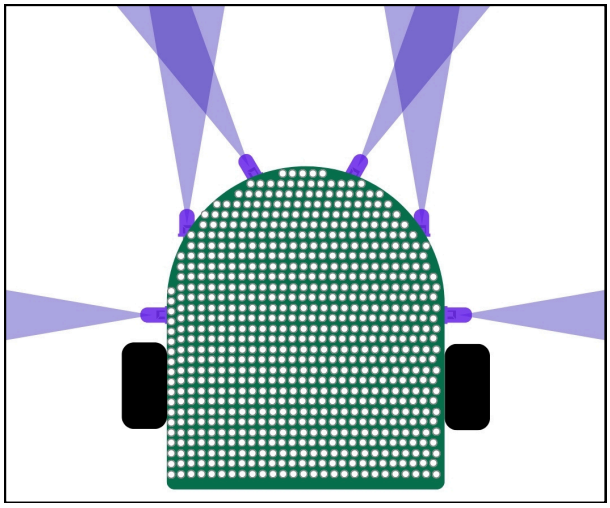
The mouse is designed to use **Floodfill** with DFS to search for the shortest route. We have designed a custom IR sensor circuit for perception that uses a simple pull up for the phototransistor while the IR sensor works on pulsed mode.

Encoders are used for memory mapping in the mouse's EEPROM. While the IMU's gyro is used for computing Yaw for turns and correct heading.

The N20 motors are powered and controlled using the motor driver.

Perception And Working

The mouse consists of 6 IR sensors each used for perception at the sides, the diagonals and at the front respectively.



The side and diagonal sensors help with the PID control of the mouse which also later helps with turn tuning.

The front sensors have been placed at the edges which helps a lot while turning the mouse smoothly.

The schematic of the mouse is as per following:

https://drive.google.com/file/d/1SF3wZ15WVjFFwZuwkKZmc4K_OCIFFX5e/view?usp=sharing

The sensors work in pulsed mode. Each IR LED is turned on for 1ms during which the phototransistor records sensor data. The IR LED is allowed to settle for 30 microseconds after which sensor data is captured and then the next sensor is called through the firing of an interrupt.

The mouse usually has a hard time when one or both walls are missing for 1-2 consecutive cells. PID values get old and the mouse has a tough time moving without drifting. Thus, to counter this the IMU is utilized along with the left or right sensor (wherever wall is present). This keeps the mouse from drifting. Diagonal sensors also help when the next wall is near the mouse

We have used silicone rubber wheels for a better grip than traditional wheels. This helps in improving the odometry of the mouse.

The working of the mouse is as per follows:

- Floodfill algorithm works on the highest or logical level of the mouse

- Sensor data is acquired and converted for distance calculation
- The mouse then selects left or right priority based on the logical input from the DIP switch
- The mouse first calibrates its sensors to the ambient light in the environment after which it goes on standby
- The mouse can then be switched on after which it starts navigating the maze

- The mouse starts navigating the cells one by one and uses a PD controller to stay in the cell without colliding.
- When one or both walls disappear, the mouse switches to gyro for steering

- The IMU also helps during turns. Turning blindly while your encoders keep drifting is not very accurate and hence, calculating yaw and turning using the IMU not only helps in turning more accurately, it also provides room for smoothening/tuning the turns

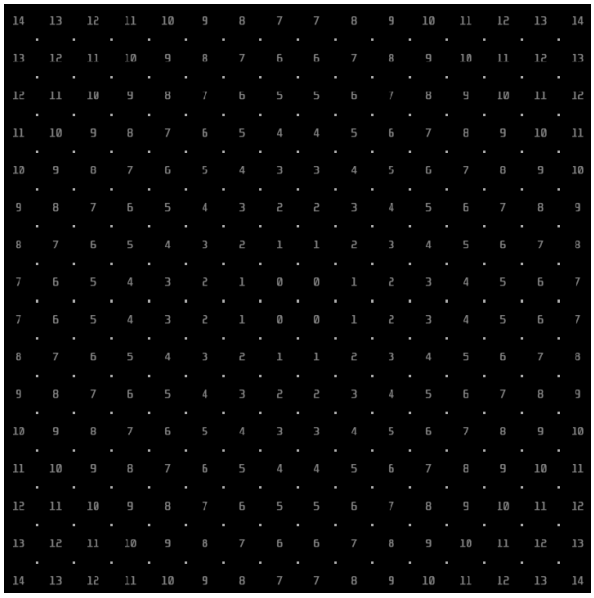
- Once the dry run is complete, the mouse computes the fastest path

- On the second run, the mouse completes the run on the smoothened path using encoder data as a feedforward controller. This allows for a more faster run than the first one.

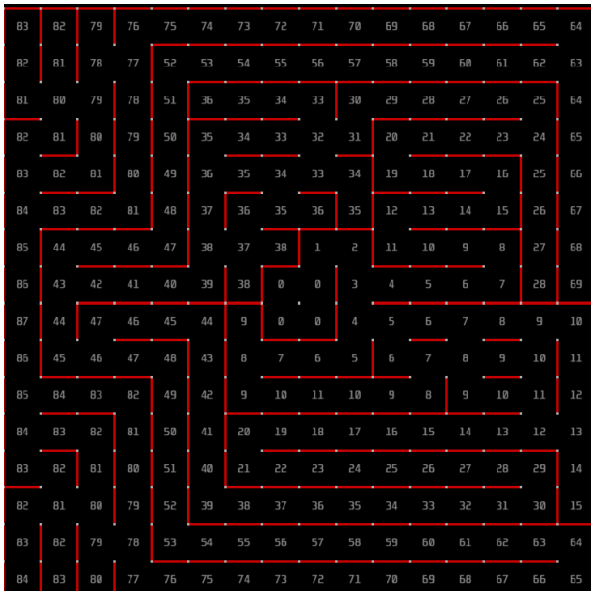
Floodfill:

- The working of the algorithm is simple. The destination cell is first selected.

- Weights are then added along the cell with successive increments in the North, South, East and West directions.
- A matrix of maze size, say for instance, 16x16 is formed.
- The mouse then starts its journey trying to move to the destination cell in decreasing order of the cell weights. On the logical level, x, y coordinates according to the 2D array or matrix is inserted as the start cell and active cell updates in the logic as per the encoder data converted into distance



cell weights on an unexplored maze



cell weights on an explored maze

- Encoder data is acquisitioned using interrupts by detecting rising and falling edges in the quadrature encoder and incrementing/decrementing by 1 each time a change is detected. This can be used to compute the rotations made by the motor. Then the distance traversed can be determined as c*N
  Where:
  c ≡ circumference of the wheel
  N ≡ No. of rotations

- Finally the turns made by the IMU also help in smoothening up the path by using algorithms like pledge and increasing weights at dead ends or cells blocked by a wall.
- Once the mouse reaches its center, it stops and is then ready for its second run on the smoothened path. Since the path is already known, it allows for faster runs at long straights and smooth turns at corridors and intersections.

Construction and Making

- The foundation of this micromouse is a double sided prototyping board with standard 2.5mm (0.1 inches) pitch through holes.
- We've implemented a 2WD delta 'wheel' configuration. The motors are placed at ⅔ length of the bot and a ball support is fixed at the underside to support its front.
- The wheels are 3d printed and fit with a silicone rubber grip. They've been carefully chamfered to not slip out during the run thus allowing for a smooth navigation through the maze.
- The wheel diameter is also set at 22mm providing a lower ground clearance and hence allowing a suction system to be implemented in our later builds for this mouse.
- The motors are mounted to the board with standard brackets, nuts and bolts.

- The six IR sensors are placed as far as possible to the motors to allow for as much time possible for the bot to react to the changes in the environment.

- The IR sensors are placed along a semicircle circumference to avoid collisions at the corners during the run.
- The IMU is placed at the geometric centre of the mouse to allow precise rotational dps (while avoiding the curvilinear translation).
- The dev board is placed with its length parallel to the prototyping board, leaving space to its right and left sides for power conversion components and motor driver.
- The battery used is a 70C 350mAh 2S LiPo. It's been placed at the very end to balance the mouse. It is the heaviest component and placing it at the back brings the center of rotation towards the motors.

- The wiring has been done on both upper and bottom layers but predominantly on the lower side to save space for components at the top.

- We've used JST connectors to connect the peripherals in order to prevent any kind of reverse connections.
- The IR LEDs (Transmitter) are controlled using MOSFETs as they can't be powered solely by the MCU.
- The IR phototransistors (receivers) are powered using 3.3V of the MCU board and their current limited by 1220 Ω resistance.

Bill of Materials (BOM):

Refer to this link for the detailed BOM
https://docs.google.com/spreadsheets/d/1L-Sz8aLauA1Dkh7gqv-jeQGjD97ZNqRHfolo6efxoAA/edit?usp=sharing

S.No.	Components	Quantity
1	WeAct Studio STM32G431CBU6	1
2	N20 Motors (with encoders)	2
3	ISM330DHCX	1
4	2S LiPo CHNL Battery Pack	1
5	TB6612FNG Motor Driver	1
6	Mini 360 Buck convertor	1
7	SFX1308 Boost convertor	1
8	IR LEDs (Vishay TSAL6100)	6
9	Phototransistors (SFH309)	6
10	DIP Switch	1
11	Mosfets (SI2300HXY)	6
12	Motor Mounts and Wheels	2
13	Double Sided PCB Prototype Board	1
14	Wires, Resistors, Headers Pins, JST Connectors etc.	--